

5-dataset-reduction

1 Dimensionality Reduction: first steps

```
[ ]: import polars as pl

PARQUET_PATH = "../../../data/raw/Waveform_tuning.parquet"
OUTPUT_PATH = "../../../data/interim/collapsed_waveform_tuning.parquet"

GROUP_KEYS = ["Color$", "HeadIdx#", "V", "F_r", "dt2", "Coverage#"]

lf = pl.scan_parquet(PARQUET_PATH)
schema = lf.collect_schema()

value_cols = [c for c in schema if c.startswith("Value_")]
```

1.0.1 Verification of unique dt2 values per group

```
[ ]: (
    lf
    .group_by(["Color$", "HeadIdx#", "V", "F_r", "Coverage#"])
    .agg(pl.col("dt2").n_unique().alias("dt2_unique"))
    .select(pl.col("dt2_unique").value_counts())
    .collect()
)
```

```
[ ]: (
    lf
    .group_by(["Color$", "HeadIdx#", "V", "F_r", "Coverage#"])
    .agg(pl.col("dt2").unique())
    .limit(5)
    .collect()
)
```

Counting how many instances have the expected 6 sheets.

```
[ ]: (
    lf
    .group_by(["Color$", "HeadIdx#", "V", "F_r", "Coverage#"])
    .len()
)
```

```

        .select(pl.col("len").value_counts())
        .collect()
    )

```

1.1 Building Data Pipeline

To build a clean data pipeline, the null columns will also be dropped, and the 6 sheets will be collapsed into one mean calculation.

```

[ ]: agg_exprs = []
    for col in value_cols:
        agg_exprs.append(pl.col(col).mean().alias(f"{col}_mean"))

    group_counts = (
        lf.group_by(GROUP_KEYS)
        .len()
        .rename({"len": "n"})
    )

    collapsed = (
        lf.join(group_counts.select(GROUP_KEYS), on=GROUP_KEYS, how="inner")
        .group_by(GROUP_KEYS)
        .agg(agg_exprs)
    )

    drop_cols = (
        [f"Value_{i:04d}#_mean" for i in range(1120, 1136)]
    )

    collapsed = collapsed.drop(drop_cols)

    collapsed.sink_parquet(OUTPUT_PATH, compression="snappy")

```

```

[ ]: print(pl.scan_parquet(OUTPUT_PATH).head(15).collect())

```

```

[ ]: df = pl.read_parquet(OUTPUT_PATH)

    value_cols = [c for c in df.columns if c.startswith("Value_")]
    feature_cols = [c for c in df.columns if c not in value_cols]

    print("Feature cols:", feature_cols)
    print("N features:", len(feature_cols))
    print("N targets:", len(value_cols))
    print("N rows:", len(df))

```

```
[ ]: # 1. How many rows total?
print(df.shape)

# 2. Confirm the waveform columns
waveform_cols = [c for c in df.columns if 'Value' in c]
print(f"Waveform columns: {len(waveform_cols)}")
print(f"Metadata columns: {[c for c in df.columns if 'Value' not in c]}")
```

2 Next Steps: Core Model

For the core model of this assignment (designing a model that will predict the colour (SD value, between 0 and 1) given a set of parameters).

This work will be continued in a [separate notebook](#).

3 Extension Model: Waveform Tuning

3.1 Compressing Nozzle Values

According to the nozzle analysis done in [EDA-chips-nozzles.ipynb](#), there might be a subset of outlier nozzles worth dropping from the dataset. The nozzles with the highest variance will be identified and dropped.

For this, we need to identify an acceptance threshold per chip, which means 4 colors x 30 chips = 120 thresholds to be calculated. To reduce computational load, this will be done on a per-colour basis. Therefore we will split the current dataset into K, C, Y, M.

This work will continue in four notebooks, one for each colour.

```
[ ]: for color in ["C", "M", "Y", "K"]:
    out_path = f"../../data/interim/colour/collapsed_{color}.parquet"
    (
        lf.filter(pl.col("Color$") == color)
        .sink_parquet(out_path, compression="snappy")
    )
    print(f"Saved {color} → {out_path}")
```